

Sheth L.U.J & Sir M.V. College Of Science

Subject :- Artificial Intelligence

Practical no 1

Aim :-

- A) Implement the Breadth First Search algorithm to solve a given problem.
- B) Implement the Iterative Depth First Search algorithm to solve the same problem.
- C) Compare the performance and efficiency of both algorithms.

Description :-

This program represents a maze as a **graph using an adjacency list** and implements two search algorithms, **Breadth-First Search (BFS)** and **Iterative Deepening Depth-First Search (IDDFS)**, to find the path from the **Start** room to the **Cheese** room. In BFS, a **deque** is used as a queue, and the algorithm explores the maze level by level while maintaining a **visited** set to avoid repeated nodes. Each path is stored and extended by adding unvisited neighboring rooms. When the **Cheese** node is reached, the complete path is returned. The IDDFS implementation uses a **Depth-Limited Search (DLS)** function that recursively explores nodes up to a specified depth. The depth limit is gradually increased from 0 up to the maximum depth, allowing the algorithm to search deeper in each iteration while avoiding cycles by checking whether a node is already present in the current path. Finally, both algorithms are executed starting from **Start**, and they find the path:

Code & input :-

```
maze_graph = {
    'Start': ['A', 'B'],
    'A': ['Start', 'C', 'D'],
    'B': ['Start', 'E'],
    'C': ['A'],           # Dead end
    'D': ['A', 'F'],
    'E': ['B', 'G'],
    'F': ['D', 'Cheese'], # Path to the cheese!
    'G': ['E'],           # Dead end
    'Cheese': ['F']
}
```

Name :- Vinod Manohar Mali

Roll No :- T092

Sheth L.U.J & Sir M.V. College Of Science
Subject :- Artificial Intelligence
Practical no 1

```
11 from collections import deque

def bfs_find_cheese(graph, start, target="Cheese"):
    queue = deque([start])
    visited = set([start])

    print("BFS Exploration Order:")
    while queue:
        path = queue.popleft()
        current_room = path[-1]
        print(f"Checking room: {current_room}")

        if current_room == target:
            return path

        for neighbor in graph.get(current_room, []):
            if neighbor not in visited:
                visited.add(neighbor)
                new_path = list(path)
                new_path.append(neighbor)
                queue.append(new_path)

    return None #

bfs_path = bfs_find_cheese(maze_graph, 'Start')
print(f"\nBFS Result Path: {bfs_path}")

BFS Exploration Order:
Checking room: Start
Checking room: A
Checking room: B
Checking room: C
Checking room: D
Checking room: E
Checking room: F
Checking room: G
Checking room: Cheese
BFS Result Path: ['Start', 'A', 'D', 'F', 'Cheese']
```

```
11 def depth_limited_search(graph, current_room, target, limit, path):
    if current_room == target:
        return path

    if limit <= 0:
        return None

    for neighbor in graph.get(current_room, []):
        if neighbor not in path:
            new_path = list(path)
            new_path.append(neighbor)

            result = depth_limited_search(graph, neighbor, target, limit - 1, new_path)

            if result is not None:
                return result

    return None

def iddfs_find_cheese(graph, start, target="Cheese", max_depth=10):
    print("IDDFS Iterations:")
    # Iteratively increase the depth limit
    for depth in range(max_depth):
        print(f"--- Searching with depth limit: {depth} ---")

        result = depth_limited_search(graph, start, target, depth, [start])
        if result is not None:
            return result

    return None # No path found within max_depth

# Run IDDFS
iddfs_path = iddfs_find_cheese(maze_graph, 'Start')
print(f"\nIDDFS Result Path: {iddfs_path}")
```

```
11 IDDFS Iterations:
--- Searching with depth limit: 0 ---
--- Searching with depth limit: 1 ---
--- Searching with depth limit: 2 ---
--- Searching with depth limit: 3 ---
--- Searching with depth limit: 4 ---

IDDFS Result Path: ['Start', 'A', 'D', 'F', 'Cheese']
```

Name :- Vinod Manohar Mali
Roll No :- T092